

# Aesthenda Master Project Instructions

A thoughtful, reliable booking system.  
A clear standard for building it well.

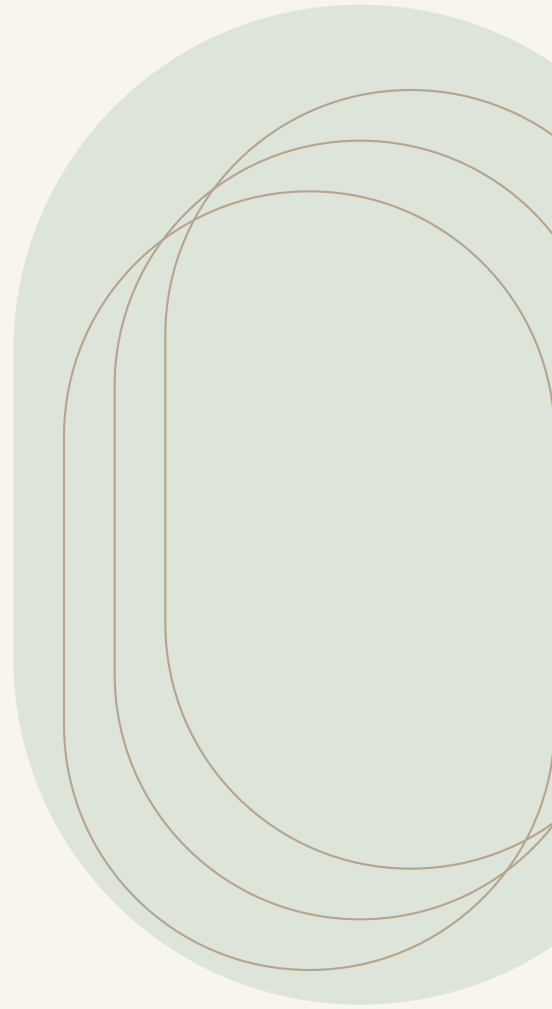
---

## A PRACTICAL GUIDE FROM PURPOSE TO HANDOFF

Product principles • Architecture • Scheduling • Experience  
Security • Delivery standards • Testing • Project continuity

VERSION 1.0 | SEPTEMBER 8, 2026

*Proposed working standard for the Aesthenda team*



# Build trust into every booking

A shared working agreement for the person shaping Aesthenda and the person doing the heavy lifting.

Aesthenda is a booking system for independent beauty and service professionals. It should make managing availability, services, clients, and appointments feel intuitive, polished, calm, and dependable. Success means a provider can trust the calendar and complete everyday work without unnecessary effort.

## Principles, in working order

- 1. Reliability.** Preserve working workflows. A beautiful interface cannot compensate for a lost appointment or incorrect availability.
- 2. Simplicity.** Choose the smallest understandable solution that solves the real problem. Avoid speculative features and unnecessary configuration.
- 3. Ease of use.** Use providers' language, make the next step obvious, and reduce repeated entry.
- 4. Data integrity and security.** Protect client information and organization boundaries. Treat these as release requirements, regardless of other priorities.
- 5. Maintainability.** Keep business rules clear, changes focused, and decisions documented so another contributor can continue confidently.
- 6. Visual consistency.** Use a coherent system of typography, spacing, components, and interaction patterns.
- 7. Future scalability.** Leave room to grow when it costs little today; add complexity only for a demonstrated need.

### How to use this guide

Start with the principles and collaboration rules. Use the relevant technical section during implementation, the completion checks before release, and the handoff template at the end of each session.

Status: proposed master working standard, version 1.0. The original draft establishes purpose, priorities, understanding before changing, and protecting existing workflows. Technical details below are recommended design rules based on the requested stack; no repository, schema, or live application was inspected. Confirm names and policies against the implementation before adopting changes.

# Make decisions with context

Give the builder room to solve problems while keeping product direction and consequential choices visible.

## Understand before changing

Before modifying a feature, identify its purpose, current behavior, dependencies, and failure modes. Follow the existing workflow through the interface, application logic, and database. Investigate ambiguity before making assumptions; do not replace working functionality solely because another implementation appears cleaner.

## Agree on the outcome

For each meaningful task, record the user problem, intended behavior, acceptance criteria, affected workflows, and what is outside scope. Use a concrete example: “A front-desk user can reschedule within the same organization, and the system rejects a conflicting slot.”

| Decision                  | Working rule                                                                                                                                                                           |
|---------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Routine implementation    | The builder chooses reversible details consistent with the existing architecture and explains material tradeoffs.                                                                      |
| Product behavior or scope | The product owner resolves business-policy choices, competing priorities, and changes to the agreed user experience.                                                                   |
| Consequential changes     | Review a concrete plan before destructive data changes, permission expansion, major schema replacement, new recurring costs, or production actions outside the agreed release process. |
| Unclear requirement       | Investigate first. Present the smallest useful question with a recommendation, impact, and a safe interim path.                                                                        |

## Keep communication useful

Report what changed, why it matters, what was verified, and what remains uncertain. Flag blockers early with evidence and options. Clearly distinguish observed behavior, assumptions, recommendations, and completed work. Never describe an untested change as verified.

## Respect the partnership

Assume good intent. Raise concerns directly and kindly. Protect focused work by limiting parallel priorities, grouping feedback, and avoiding silent scope growth. Record decisions once in a shared decision log so neither person must repeatedly reconstruct the conversation.

### Default habit

Leave the project easier to understand than you found it. Preserve the owner’s existing work and explain any change that alters an established workflow.

# One organization boundary

Recommended conceptual model for Supabase. Adapt it to the existing schema; these are logical entities, not verified table names.

**Responsibility flow**

Interface → application services → Supabase Auth / PostgreSQL.  
The interface guides the user. Trusted application logic coordinates workflows. The database enforces relationships, permissions, and booking invariants.

| Entity                       | Responsibility and relationships                                                                                                                                                                       |
|------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Organizations                | The tenant boundary for each business. Owns settings, timezone, clients, providers, services, and appointments.                                                                                        |
| Users & memberships          | Supabase Auth identifies the user. Memberships connect a user to an organization and its role. A user may have different roles in different organizations.                                             |
| Providers                    | A bookable professional within an organization. Keep the provider record distinct from the login account; link them explicitly when the provider has access.                                           |
| Services & provider services | Services belong to the organization. A joining relationship declares which providers offer each service, with explicit duration or price overrides only if supported.                                  |
| Availability & time off      | Recurring working hours, dated exceptions, and blocked time belong to the relevant provider and organization.                                                                                          |
| Clients & appointments       | Appointments reference an organization, client, provider, and selected service or service items. Store the booked duration and price snapshot so later catalog edits do not silently rewrite bookings. |

**Relationships must remain inside the organization**

Every tenant-owned record must have an unambiguous organization scope. Enforce same-organization references through database constraints or equally strong validated database logic. A valid client ID or provider ID is insufficient if it belongs to another organization.

**Keep the boundaries explicit**

Centralize booking rules, time calculations, and role checks. Use migrations for schema changes, constraints, and policies. Keep secrets and privileged operations on trusted servers. Add Storage, Realtime, background jobs, or payments only when a defined workflow needs them.

Technical reference: [Supabase: Tables and Data](#). Organization design and entity relationships above are project recommendations.

# Permissions follow the work

Proposed minimum-access baseline. Confirm this matrix before implementing or expanding access.

| Capability                             | admin              | front_desk            | stylist                             |
|----------------------------------------|--------------------|-----------------------|-------------------------------------|
| Organization settings, team and roles  | Manage             | No                    | No                                  |
| Services and provider assignments      | Manage             | Read                  | Read assigned                       |
| Appointments                           | Organization-wide  | Organization-wide     | Own assigned only                   |
| Client information                     | As needed for work | As needed for booking | Minimum needed for own appointments |
| Availability and time off              | Manage team        | Read; request changes | Own, if enabled                     |
| Bulk exports or destructive operations | Explicit policy    | No by default         | No by default                       |

“Manage appointments” means permitted creation, editing, rescheduling, and cancellation; it does not grant hard deletion or a conflict override. An organization admin is not a platform administrator. Public booking, if supported, requires a separate, narrowly scoped access path.

## Enforce access at the database boundary

Enable row-level security (RLS) on exposed tables and review table grants. Policies must validate active organization membership and permitted operations. Test reads, inserts, updates, and deletes. Prevent a user from changing organization ownership or assigning themselves a stronger role. Hiding a button is not authorization. [1]

## Keep identity and privileges separate

Use trusted membership records for business roles; do not treat user-editable profile metadata as authority. Supabase roles such as authenticated are different from Aesthenda roles such as stylist. Privileged service credentials can bypass RLS and must stay server-side. Review views and privileged functions for accidental bypasses. [1]

## Handle membership changes cleanly

Switching organizations must clear stale tenant data and reload access. Removing a team member must remove their access without deleting appointment history. Test invitations, expired sessions, revoked memberships, and users who belong to multiple organizations.

[1] [Supabase: Row Level Security](#). The permission matrix is a recommended Aesthenda policy, not a statement of existing access.

# The calendar must be trustworthy

Use one documented definition of a bookable slot across creation, editing, rescheduling, and public booking.

## Eligibility comes before availability

A booking must use an active organization, an eligible provider, and a service that provider offers. Apply the resolved service duration and any required buffers. Check working hours, time off, existing blocking appointments, lead time, and booking horizon. Check shared rooms or equipment only if those resources are part of the product.

## Define time precisely

Store appointment instants with timezone-aware timestamps and keep an IANA timezone for each organization, such as America/New\_York. Define recurring hours in local time and resolve each date carefully. Show the timezone when it could be ambiguous. Handle daylight-saving gaps and repeated times explicitly; never silently shift the client’s chosen time.

### Default overlap rule

Represent occupied time as [start, end), including required buffers. Two bookings conflict when  $\text{new\_start} < \text{existing\_end}$  AND  $\text{new\_end} > \text{existing\_start}$  for the same provider. Adjacent appointments are allowed only when their occupied intervals do not overlap.

| Example                              | Expected behavior                                    |
|--------------------------------------|------------------------------------------------------|
| 10:00-11:00, then 11:00-12:00        | Allowed when there are no buffers.                   |
| 10:00-11:00 with 15-minute cleanup   | The next booking cannot occupy time before 11:15.    |
| 10:30-11:30 on the same provider     | Reject if a blocking 10:00-11:00 booking exists.     |
| Same interval on different providers | Allowed unless a shared required resource conflicts. |

## Write down which statuses occupy time

Recommended baseline: pending confirmation and confirmed bookings reserve time; an active temporary hold reserves time until expiry. Cancellation releases future capacity. Keep completed and no-show history intact and define their occupancy treatment explicitly. Do not invent holds or statuses if the existing product does not need them.

Range-boundary reference: [PostgreSQL: Range Types](#). Scheduling policies here are recommended defaults.

# Protect the final save

A slot shown as available is a candidate. It becomes a booking only after a successful authoritative save.

## Prevent races, not just visible overlaps

Two people can select the same available slot before either saves. Make conflict validation and persistence atomic. For a strict one-booking-per-provider model, consider a PostgreSQL range exclusion constraint scoped by organization and provider. Include buffers and blocking statuses. More complex capacity rules need a carefully locked transaction or equivalent database guarantee.

Every write path must obey the same rule: staff calendar, public booking, imports, integrations, edits, and rescheduling. A browser-only check or a separate “check then insert” request cannot guarantee conflict prevention.

## Make edits safe

Exclude the appointment itself when checking a reschedule. Revalidate eligibility, duration, buffers, and conflicts when the provider, service, time, or status changes. If the new slot fails, preserve the original booking. Detect stale edits so one person does not silently overwrite another person’s changes.

## Make retries predictable

Use a unique request identifier or another idempotent design for booking creation and external callbacks. A double-click, network retry, or repeated webhook must not create a duplicate appointment. Show success only after persistence is confirmed; on an uncertain response, reconcile the request before inviting a second submission.

## Keep notifications consistent with reality

Send confirmations and reminders only from committed booking state. Track failures and retries separately so a failed message does not erase a valid appointment. Make cancellation and rescheduling stop or update obsolete reminders. If payments exist, verify their external events and define recovery for partial failures.

### Double-booking policy

Default to no overlapping occupied time for a provider. If the business needs intentional overlap, approve an explicit capacity model or limited override policy with authorized roles, a reason, audit history, and tests. A generic “ignore conflict” button is not a complete policy.

Acceptance example: two simultaneous requests for the same provider and slot result in one confirmed save and one clear conflict response, with no duplicate records or notifications.

Constraint reference: [PostgreSQL: Constraints on Ranges](#). Select the mechanism after reviewing the actual booking model.

# Get to the first useful booking

Ask for what is needed now. Make the remaining setup visible, optional where appropriate, and easy to resume.

## A recommended setup path

- 1. Establish the business.** Create the organization and its first admin through a protected flow. Capture the business name and timezone; explain why the timezone matters.
- 2. Add a provider.** Support a solo professional without forcing team administration. Keep a bookable provider distinct from a login invitation.
- 3. Add a service.** Capture a clear name, duration, and applicable price. Assign at least one eligible provider. Explain what clients will see.
- 4. Set availability.** Collect working hours, breaks or buffers, and exceptions. Preview the result in the same local timezone used by the calendar.
- 5. Preview and test.** Guide the user through a test booking and its edit or cancellation. Make test records distinguishable and easy to remove safely.
- 6. Publish when ready.** If public booking exists, require complete bookability settings before enabling it. Present a concise readiness checklist and a preview of the booking experience.

## Make progress durable

Save completed steps so refreshes, sign-outs, and invitations do not restart setup. Derive readiness from the underlying records, not only a “completed” flag. Repeated setup attempts must not create duplicate organizations, providers, or services.

## Use helpful empty states

Explain what is missing, why it matters, and the next action: “Add a service before opening online booking.” Keep advanced policies and optional integrations out of the critical path. Do not seed production with realistic-looking fake clients or appointments.

### Onboarding is complete when

An authorized user can offer a real service with an eligible provider and valid availability, create a booking, understand where it appears, and return later without losing setup progress.

Evaluate with representative providers: where do they hesitate, ask for help, abandon setup, or misinterpret a calendar result? Use observed friction to simplify the flow.

# Calm, clear, and consistent

A restrained spa-neutral aesthetic should support legibility and confidence in daily work.

## Build a small visual system

Use warm off-white backgrounds, muted sage or stone surfaces, dark readable text, and a restrained accent color. Define reusable spacing, typography, border, radius, and elevation tokens. Reserve decorative type for short headings; keep operational text, times, and labels easy to scan.

## Let the work lead the screen

Give each view a clear purpose and a primary action. Make the calendar’s date, provider, timezone, and current filters visible. Keep appointment details consistent across calendar cards, lists, dialogs, and confirmations. Use familiar words such as “client,” “service,” and “appointment.”

| State                   | Required experience                                                                                                |
|-------------------------|--------------------------------------------------------------------------------------------------------------------|
| Loading                 | Show that work is underway; prevent duplicate submission while retaining the user's context.                       |
| Empty                   | Explain the state and offer the relevant next step.                                                                |
| Validation error        | Identify the field and explain how to fix it. Preserve entered information.                                        |
| Save or network failure | Keep the form usable and state whether the save is known to have failed or still needs reconciliation.             |
| Success                 | Confirm what changed and reflect the saved record in the calendar.                                                 |
| Destructive action      | Name the record and consequence. Confirm when the impact warrants it; prefer reversible cancellation or archiving. |

## Make every core path accessible

Support keyboard navigation, visible focus, meaningful labels, sensible tab order, and readable contrast. Do not rely on color alone for status or availability. Provide a non-drag alternative to moving appointments. Keep touch controls usable and dialogs accessible on small screens.

## Check real content, not only ideal examples

Review long service names, dense calendars, large text, narrow screens, empty schedules, multiple providers, and validation messages. Visual polish is incomplete if the interface clips real data or hides a failed booking behind a reassuring message.

# Protect people and records

Build safeguards into normal workflows and verify them with realistic failure cases.

## Collect and expose only what is needed

Limit client fields to a clear business purpose. Keep private notes separate from public booking data. Avoid client details, credentials, tokens, and payment information in screenshots, logs, analytics, or test fixtures. Define retention, export, and deletion rules with the owner before implementing those workflows.

## Keep production access deliberate

Separate development, staging, and production configuration. Use synthetic or appropriately de-identified test data. Store secrets in approved secret management, not source code or shared documents. Record who owns each account and how access is granted and revoked. Rotate exposed secrets promptly.

## Enforce valid data at the source

Use primary keys, foreign keys, required fields, valid status values, positive durations, and appropriate uniqueness rules. Validate inputs at trusted boundaries. Prevent cross-organization relationships. Store money using integer minor units or a suitable fixed-precision type, with currency, if prices are supported.

## Preserve history and trace meaningful actions

Archive services and providers when existing appointments reference them. Snapshot booking terms where necessary. Record the actor, organization, time, target, and reason for sensitive changes such as role changes, cancellations, overrides, and exports. Protect audit records from ordinary edits and avoid copying unnecessary personal data into them.

## Plan recovery before it is needed

Confirm which database and file backups exist, who can restore them, and the agreed recovery objectives. Test restoration in an isolated environment. Before risky migrations, document the recovery path; do not assume reverting application code will reverse a data transformation.

### Incident response

Contain the exposure or unsafe write path, preserve useful evidence without spreading private data, notify the responsible owner, restore or correct data through a reviewed process, and record the cause and preventive action.

Before release, demonstrate that a user in Organization A cannot read, create, change, or delete Organization B's records, including by direct requests and guessed identifiers.

# Keep one authoritative project

Use the existing repository conventions unless a documented change solves a real maintenance problem.

## Organize around understandable responsibilities

Separate interface components, feature workflows, domain rules, data access, and environment configuration. Keep scheduling and authorization logic centralized. Use clear names, small cohesive functions, consistent formatting, and explanatory comments for non-obvious business decisions. Avoid broad cleanup mixed into an unrelated feature.

| Source                      | Authority and maintenance rule                                                                                                                                 |
|-----------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Repository & default branch | Source of truth for reviewed application code. Work in focused branches or the agreed equivalent; identify changes by commit, not by a folder called “latest.” |
| Database migrations         | Source of truth for intended schema, constraints, and access policies. Commit changes and verify the applied migration history in each environment.            |
| Deployed release            | Record the exact commit or release tag, environment, deployment time, and migration state. A deployment is not automatically the newest local code.            |
| Project documentation       | Keep setup, decisions, schema notes, and known issues versioned with the code where practical.                                                                 |
| Design files & exports      | Name the canonical design location. Date or version review exports and mark superseded copies; exported PDFs are reference snapshots.                          |
| Secrets & local settings    | Store outside version control. Maintain an example configuration with names and descriptions, never working credentials.                                       |

## Avoid silent drift

Do not overwrite uncommitted work or promote an old archive without comparing it to the canonical branch. Before changing files, identify the repository, branch, environment, and current status. Avoid unexplained copies such as `final_v2_really_final`. Keep generated outputs and temporary experiments separate from production source.

## Make changes reproducible

Commit dependency lockfiles. Add migrations instead of rewriting migration history that has already been shared or applied. Reconcile emergency dashboard edits into reviewed migrations and notes promptly. Include the reason, affected behavior, and relevant checks in each change description.

# Finish the whole workflow

A feature is done when it works for the intended user, protects adjacent behavior, and can be maintained.

- **Outcome:** Acceptance criteria are met, including the normal path and relevant failure states. Any scope change is documented.
- **Workflow:** The feature works end to end with persisted data. Refreshing or signing back in shows the correct result.
- **Permissions:** Allowed roles can complete the task; unauthorized roles and other organizations cannot.
- **Data:** Validation, database relationships, status transitions, concurrency behavior, and historical records remain correct.
- **Experience:** Loading, empty, error, and success states are implemented. Keyboard and mobile use are checked where relevant.
- **Regression:** Affected surrounding workflows have passed the appropriate checklist. Failures are fixed or explicitly block release.
- **Code:** The change follows project conventions, removes temporary debug behavior, and passes applicable build, lint, type, and automated checks.
- **Operations:** Migrations, configuration, external dependencies, monitoring, and recovery steps are understood and recorded.
- **Handoff:** Documentation, known issues, test evidence, and next steps are current. The exact version and environment are identifiable.

## Keep implementation and release status separate

Use clear states: in progress, ready for review, verified in staging, and released. A local demonstration does not mean production is updated. If a check was not run, label it “not run” with the reason and follow-up owner.

## Release with evidence

Review the final change against acceptance criteria, apply migrations through the agreed process, and perform a short smoke check in the target environment. Confirm login, organization scope, calendar loading, and an appropriate safe booking flow. Observe errors after release and keep the recovery path available.

### Stop-release conditions

Do not release known cross-organization exposure, unauthorized access, corrupted booking data, unresolved double-booking races, or a broken core booking path. Cosmetic polish can be scheduled; these failures need resolution.

# Prove the behavior that matters

For each release, record pass, fail, not run, or not applicable, with environment, version, and evidence.

| Check                    | Minimum expected evidence                                                                                                                          |
|--------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------|
| Identity & organizations | Login/logout and session expiry behave correctly; organization switching clears old data; revoked or foreign memberships deny access.              |
| Roles & RLS              | Exercise admin, front_desk, stylist, signed-out access, and a second organization. Verify permitted and forbidden direct reads and writes.         |
| Provider / service setup | Only eligible active provider-service combinations can be booked. Overrides resolve consistently; catalog edits preserve existing booking terms.   |
| Create / edit / cancel   | Create, refresh, edit, reschedule, and cancel. Confirm client, duration, provider, status, released capacity, and calendar display.                |
| Time boundaries          | Test adjacency, buffers, partial overlap, full containment, time off, working-hour edges, midnight, and daylight-saving changes.                   |
| Concurrent requests      | Submit the same slot simultaneously. Only the allowed capacity succeeds. Retry the same creation request and confirm no duplicate.                 |
| Failure recovery         | Test interrupted saves, double-clicks, stale edits, failed notifications, and expired holds if supported. Preserve or reconcile the correct state. |
| Onboarding               | A new organization reaches its first booking. Resume incomplete setup. Repeat steps without duplicating records.                                   |
| UI & accessibility       | Check mobile and desktop, keyboard operation, visible focus, long labels, dense calendars, and every major loading/error state.                    |
| Migrations & release     | Apply migrations to a representative test database; verify existing data, access policies, deployed version, and target-environment smoke checks.  |

## Match the test to the risk

Use unit tests for time calculations and pure rules; database or integration tests for permissions, constraints, and transactions; end-to-end tests for core user journeys. Add a regression test for a reproducible defect when it protects meaningful behavior.

If payments, reminders, resource capacity, imports, or public booking are enabled, add targeted checks for their actual flows. Keep unavailable features marked not applicable rather than claiming they passed.

# Leave a clear path forward

Document what the next person needs to operate the project, then prioritize the smallest useful next outcome.

## Minimum handoff package

**Setup:** repository and branch, runtime requirements, installation and run instructions, configuration names, and synthetic test accounts.

**System:** entity relationships, organization boundary, roles matrix, RLS intent, booking rules, and migration process.

**Operations:** environment locations, release procedure, account owners, monitoring, backups, recovery steps, and third-party dependencies.

**Continuity:** decision log, current roadmap, known issues, test evidence, and the latest end-of-session note.

## Maintain documents when behavior changes

Update the relevant documentation in the same change as a schema, policy, workflow, or setup change. Explain the reason behind non-obvious choices. Record known limitations with their impact, workaround, next action, and owner. Never include working secrets in handoff materials.

| Priority               | Focus and exit condition                                                                                                                       |
|------------------------|------------------------------------------------------------------------------------------------------------------------------------------------|
| 1 / Foundation         | Confirm organization isolation, access, schema integrity, and recovery. Exit when boundary and integrity checks pass.                          |
| 2 / Reliable booking   | Complete provider/service setup, availability, booking, editing, cancellation, and conflict prevention. Exit when core regressions pass.       |
| 3 / Usable launch      | Simplify onboarding, improve accessibility, polish the calendar, and document support. Exit when representative users complete the core tasks. |
| 4 / Measured expansion | Add reminders, payments, reporting, or advanced scheduling only for a validated need with defined acceptance criteria.                         |

## Choose work by impact and dependency

Prioritize security and data loss first, then broken bookings, frequent user friction, and useful improvements. Compare reach, severity, confidence, effort, and prerequisite work. Keep active work limited; put new ideas in the backlog instead of silently expanding an in-progress task.

### A useful roadmap item

Name the user problem, desired outcome, acceptance criteria, dependencies, owner, and success signal. Measure booking failures, conflict prevention, setup completion, and task friction without collecting unnecessary client data.

# Make the next start easy

Copy this template into the project's canonical session log. Keep it concise, factual, and specific.

| Field                 | Complete at the end of the session                                                  |
|-----------------------|-------------------------------------------------------------------------------------|
| Session               | Date / contributor / task or outcome                                                |
| Version & environment | Repository / branch / commit or uncommitted files / environment / deployment status |
| Completed             | What changed and why; links to the relevant files, change, or decision              |
| Verified              | Checks run, results, and evidence; identify checks not run                          |
| Data & configuration  | Migrations applied or pending; configuration changes; no secret values              |
| Open issues & risks   | What remains uncertain or broken, its impact, and any workaround                    |
| Decisions needed      | The exact unresolved choice, recommendation, and responsible person                 |
| Next action           | The single best next step, followed by any dependencies                             |
| Resume safely         | Setup notes, local work to preserve, and recovery instructions if relevant          |

## Adoption decisions to settle first

Confirm the actual schema and canonical repository; agree on the role matrix; define booking statuses, buffers, timezone behavior, and any double-booking exceptions; identify the release owner and recovery expectations. Record each choice in the decision log. These are adoption tasks, not claims that the existing project is missing them.

## Basis and technical references

Prepared from the available "Aesthenda Project Instructions" conversation and the requested master-guide scope. The available draft ends during the surrounding-workflow checklist. This guide expands that draft; it does not certify the state of the application.

Technical references checked September 8, 2026:

1. [Supabase - Row Level Security](#)
2. [Supabase - Tables and Data](#)
3. [PostgreSQL - Range Types and Constraints](#)